

Lesson 2 — HTTP Fundamentals

The Underlying Protocol of RESTful Architecture

Because REST relies heavily on the semantics of the underlying protocol, it is impossible to design a robust REST API without a deep understanding of HTTP (Hypertext Transfer Protocol). The rules of HTTP dictate how clients and servers should behave, how networks cache data, and how failures are communicated.

1. The Structure of HTTP Messages

Communication in HTTP happens via discrete text-based messages: Requests (from the client) and Responses (from the server).

HTTP Request Structure

- **Request Line:** Contains the HTTP Method, the Request Target (URL/URI), and the HTTP Version. (e.g., `GET /api/vehicles HTTP/1.1`)
- **Headers:** Key-value pairs providing metadata about the request.
- **Empty Line:** Separates headers from the body.
- **Body (Optional):** The data being sent to the server (e.g., a JSON payload).

HTTP Response Structure

- **Status Line:** Protocol Version, Status Code, and Status Text. (e.g., `HTTP/1.1 200 OK`)
- **Headers:** Metadata about the response.
- **Empty Line:** Separates headers from the body.
- **Body (Optional):** The data returned by the server.

Critical HTTP Headers

Two headers dictate how data is formatted and negotiated between client and server:

- **Content-Type:** Describes the format of the data in the *current* message's body (e.g., `application/json`). If a client sends a POST request with JSON, it must set `Content-Type: application/json` .
- **Accept:** Sent by the client to tell the server what format it expects in the response (e.g., `application/json` or `text/xml`).

2. URI, URL, Route Parameters, and Query Strings

To access resources, clients use identifiers and locators.

- **URI (Uniform Resource Identifier):** A string that uniquely identifies a resource.
- **URL (Uniform Resource Locator):** A type of URI that also provides the means to locate the resource (includes protocol and domain, like `https://api.example.com/vehicles`).

Route Parameters vs. Query Strings

It is important to know when to put data in the URL path versus the query string.

- **Route Parameter:** Used to uniquely identify a specific resource in a hierarchy. Example: `/api/vehicles/15` (Identifies vehicle #15).
- **Query String:** Used to filter, sort, or paginate a collection of resources. It is appended after a `?`. Example: `/api/vehicles?make=Ford&sort=price`.

3. HTTP Methods Deep Dive

Before defining the methods, two crucial technical concepts must be understood (defined by RFC 9110):

- **Safe Methods:** A method is safe if it is inherently read-only and does not alter the state of the resource on the server.
- **Idempotent Methods:** A method is idempotent if executing it multiple times yields the same state on the server as executing it exactly once. (Note: The *response status* might change, e.g., 200 on first delete, 404 on second, but the server *state* remains identical).

GET

- **Semantic Meaning:** Retrieve a representation of a resource.
- **Body:** Normally does NOT have a request body.
- **Properties:** Safe and Idempotent.
- **ASP.NET Core Example:** `[HttpGet("{id}")] public IActionResult GetVehicle(int id)`

POST

- **Semantic Meaning:** Submit an entity to a specific resource, often causing a change in state or side effects on the server. Commonly used to create a *new* resource where the server assigns the ID.
- **Body:** Requires a request body.
- **Properties:** NOT Safe, NOT Idempotent (sending the same POST twice creates two distinct resources).
- **ASP.NET Core Example:** `[HttpPost] public IActionResult CreateVehicle([FromBody] CreateVehicleDto request)`

PUT

- **Semantic Meaning:** Completely replace the target resource with the request payload. If the resource does not exist, it may create it (if the client provides the ID).
- **Body:** Requires a request body representing the complete resource.
- **Properties:** NOT Safe, but IS Idempotent. (Replacing a record with the exact same data 10 times results in the same final state as doing it once).
- **ASP.NET Core Example:**

```
[HttpPut("{id}")] public IActionResult UpdateVehicle(int id, [FromBody] UpdateVehicleDto request)
```

PATCH

- **Semantic Meaning:** Apply partial modifications to a resource.
- **Body:** Requires a request body (often using JSON Patch format, describing *how* to modify the resource).
- **Properties:** NOT Safe. By strict definition, PATCH is NOT required to be idempotent, though APIs often implement it in an idempotent way.
- **ASP.NET Core Example:**

```
[HttpPatch("{id}")] public IActionResult PartiallyUpdateVehicle(...)
```

DELETE

- **Semantic Meaning:** Remove the target resource.
- **Body:** Normally does NOT have a request body.
- **Properties:** NOT Safe, but IS Idempotent.
- **ASP.NET Core Example:**

```
[HttpDelete("{id}")] public IActionResult DeleteVehicle(int id)
```

HEAD and OPTIONS

HEAD is identical to GET, but the server must NOT return a message body. It is used to check if a resource exists or to read metadata (headers) without downloading the full payload. **OPTIONS** is used by the client to find out what HTTP methods are permitted on a given URL (crucial for CORS - Cross-Origin Resource Sharing).

4. Key Semantic Comparisons and Trade-offs

PUT vs. PATCH

This is a common point of confusion. Use **PUT** when you want to replace the *entire* resource. If a vehicle has 10 fields and you send a PUT with only 2 fields, the server is expected to nullify or default the remaining 8 fields. Use **PATCH** when you only want to update specific fields without affecting the rest.

Why it matters: PUT requires the client to know and send the entire state of the object, which can waste bandwidth for large resources. PATCH is more efficient but complex to implement safely on the server side (often requiring JSON Patch document structures).

POST vs. PUT (For Creation)

Both can be used to create resources. The difference lies in who determines the URI (the ID).

- Use **POST** to `/api/vehicles` when the *server* generates the ID (e.g., auto-incrementing database column) and returns the new URI to the client.
- Use **PUT** to `/api/vehicles/500` when the *client* determines the ID (e.g., generating a GUID client-side) and wants to place the resource exactly at that URI.

DELETE vs. POST (For Removal)

In action-oriented designs, developers sometimes use `POST /api/vehicles/5/delete`. In REST, `DELETE /api/vehicles/5` is required.

Technical reasoning: HTTP infrastructure relies on methods to understand traffic. If a request is sent as POST, network intermediaries do not know it is a destructive action that should invalidate the cache for that resource. Using DELETE explicitly signals the semantic intent to the entire network path, ensuring caches are cleared and the operation is handled idempotently.